

Predicting Airbnb Listing Price Across Different Cities

Team: Jai Boonlia (PES1UG24CS197) and Kanishka Sarraf (PES1UG24CS215) | Section D | Problem 71 | UE24CS352A Machine Learning

1. Problem Statement

Airbnb hosts set nightly prices empirically, which is hard for new hosts and leaves guests unsure whether a listing is fairly priced. We build a supervised regression system that predicts the nightly listing price from listing attributes (room size, fees, reviews, location, amenities, text descriptions, dates) for New York City, Paris and Berlin. We also ask whether a model trained on several cities generalises to a city it has never seen. The work follows Luo, Zhou and Zhou (2019).

2. Dataset

Three Kaggle Airbnb listing tables (96 raw columns each): NYC (44,317 listings), Paris (59,881) and Berlin (22,552). Each city is split train:validation:test = 7:2:1. The target is the listing price in dollars per night. Features are grouped as continuous (28 after removing highly correlated ones), categorical (20 encoded features), text (12 columns such as summary, transit, neighbourhood overview) and date (host_since, first_review, last_review). Identifier-like or near-constant columns (id, host_id, market, country_code, license) were dropped.

3. Approach

- **Outliers / label:** either cap price at \$500 (removes about 1% of listings) or transform the label; square-root and logarithm both help, log is best.
- **Continuous:** 0-fill for security deposit and cleaning fee, mean-fill otherwise, standardise to $N(0,1)$; optional degree-2 interaction terms.
- **Categorical:** one-hot encoding; list fields (amenities, host_verifications) turned into multi-hot vectors through a dictionary.
- **Text:** tf-idf on unigrams and bigrams, then truncated SVD to 50 dimensions per column (600 in total).
- **Date:** null replaced by mean date, converted to days since the earliest date.
- **Models:** L2-regularised linear regression and KNN (baselines), Random Forest, XGBoost and a multilayer perceptron. Metrics are R^2 and MSE.
- **Cross-city learning:** train the network on NYC, Paris, and NYC+Paris, then test on Berlin, which is never seen during training.

4. Implementation Overview

Python repository: `config.py` holds all hyper-parameters; `src/data.py` loads and cleans the CSVs, applies the \$500 cap, splits the data and implements label transforms; `src/features.py` builds the C / I / O / T / D feature groups, fitted on the training split only to avoid leakage; `src/models.py` wraps scikit-learn and XGBoost models and a PyTorch MLP; `run_experiments.py` regenerates every table (- - table 1 . . 7) into `results/`. The final network has two hidden layers (64, 32) with ReLU, MSE loss, SGD with Nesterov momentum (lr 0.005, momentum 0.9), L2 = 0.005, and early stopping on a 10% hold-out of the training data (change in validation R^2 below $1e-4$ for 10 iterations). The smaller network and early stopping were adopted because the larger 3-hidden-layer model (128, 512, 64) overfitted.

5. Results

Reference values reported by Luo et al. (2019); full tables are in `docs/PAPER_RESULTS.md` and our own runs are written to `results/`.

Model (NYC, continuous, price <= \$500)	Train R2	Test R2	Test MSE
Linear regression (L2)	0.489	0.497	3541.99
K-nearest neighbour	0.351	0.323	4760.61
Random forest	0.752	0.655	2427.05
Neural network	0.702	0.665	2367.44
XGBoost	0.669	0.665	2357.83

XGBoost configuration	Train R2	Test R2
C, raw price (no cap)	0.558	0.195
C, sqrt label	0.652	0.537
C, log label	0.695	0.669
C + interactions, log label	0.708	0.676
Date only (TH)	0.033	0.016
Categorical only (TH)	0.546	0.532
C+O+T (TH)	0.724	0.706
C+O+T, log label	0.749	0.740

Neural network (C+O, log)	Train R2	Test R2
NYC	0.769	0.741 (NYC)
Paris	0.762	0.716 (Paris)
NYC + Paris	0.816	0.773 (combined)

Trained on -> tested on Berlin	Test MSE	Test R2
NYC only	1.78007	-3.512
Paris only	7.26637	-17.42
NYC + Paris	0.19163	0.514

6. Conclusions

- XGBoost and the neural network clearly beat the linear and KNN baselines (test R^2 about 0.665 on continuous features alone, versus 0.50 and 0.32).
- Prices are heavily right-skewed: a log transform raised XGBoost test R^2 from 0.195 to 0.669 even without removing outliers.
- Categorical and text features add substantial signal (0.665 to 0.706 / 0.740 with log label), while date features are nearly uninformative (R^2 about 0.02).
- Training on NYC + Paris gave the best network (test R^2 0.773) and was the only one that transferred to Berlin (0.514); single-city models fail badly (negative R^2), suggesting multi-city training keeps generic price drivers and suppresses city-specific ones.
- Limitations: the network overfits when the feature count is large, hyper-parameters were barely tuned, and Berlin performance (0.514) is still well below in-city performance.

Reference: Y. Luo, X. Zhou, Y. Zhou, *Predicting Airbnb Listing Price Across Different Cities*, 2019.